



PariPari

Tracciare e dividere le spese tra amici, riducendo al minimo le transazioni. Anche offline.

Documentazione di progetto

Gruppo	Ctrl+C Ctrl+V
Componenti	Fabrizi Molodduwa Quiroz Andri 928604
	Gnoni Riccardo 907420
	Scotti Nicolò 916163
Repository	github.com/AndriMolo/PariPari
Piattaforma	Android · Java · Room · Firebase · ML Kit · Material 3

Indice

Introduzione	3
Strumenti di lavoro	3
1. Funzionalità	4
1.1 Account e profilo	4
1.2 Schede e inviti	5
1.3 Dettaglio scheda	5
1.4 Spese e quote	6
1.5 Saldi, rimborsi e pagamenti	6
1.6 Notifiche	7
1.7 Convertitore valuta	7
2. Architettura software	8
2.1 Pattern architetturale (Clean Architecture / MVVM)	8
2.2 Diagramma architetturale dei layer	8
2.3 Struttura dei package	11
2.4 Gestione dati e persistenza	12
2.4.1 Database locale e caching (Room, SQLite)	12
2.4.2 Servizi cloud e Backend-as-a-Service (Firebase)	12
2.5 Networking e integrazione API esterne	13
2.5.1 Definizione API / Endpoint e DTO	13
2.5.2 Gestione asincronia, errori e resilienza di rete	13
3. Design	14
3.1 Layouts	14
3.2 Palette e tipografia	18
Conclusioni	19
Punti di forza	19
Ostacoli incontrati	19
Sviluppi futuri	20

Introduzione

PariPari è un'app Android per tracciare e dividere le spese tra amici. Quando in un gruppo — coinquilini, compagni di viaggio, colleghi — persone diverse anticipano importi diversi, alla fine ognuno si ritrova a dover restituire frazioni di denaro a più persone. PariPari tiene il conto di chi ha pagato cosa e semplifica i conti riducendo al minimo le transazioni necessarie a chiudere i debiti.

L'app si fonda su quattro scelte di progetto:

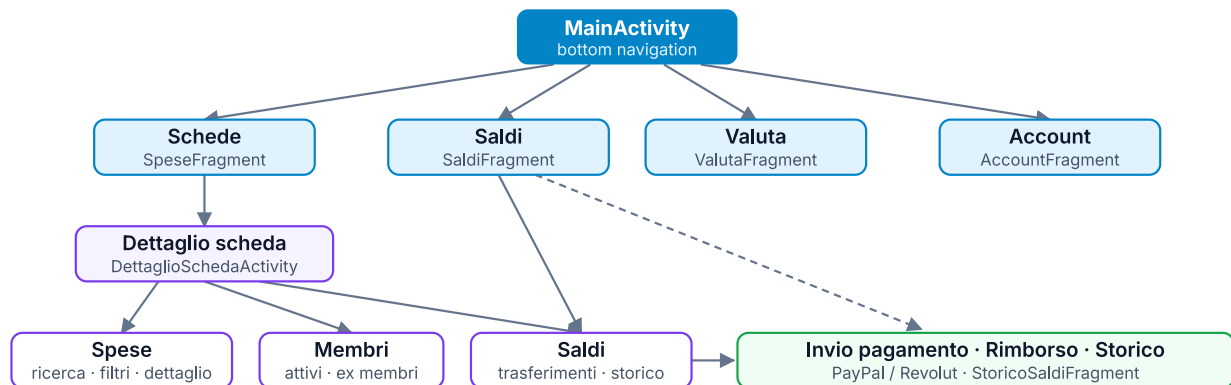
1. **Offline-first.** Ogni dato è scritto prima nel database locale Room; la sincronizzazione con Cloud Firestore avviene in background quando rete e account sono disponibili, e gli altri membri ricevono una notifica a ogni movimento.
2. **Pareggio dei saldi.** Le spese, anche in valute diverse, vengono ricondotte alla valuta del gruppo; un algoritmo greedy trasforma i bilanci netti nell'elenco dei rimborsi da fare: al più $N - 1$ per N membri.
3. **Inserimento senza attrito.** Scansione OCR dello scontrino, calcolatrice inline nell'importo, categoria suggerita dal titolo, divisione equa, per percentuali o per parti.
4. **Pagamenti in un tap.** I debiti si saldano con link PayPal.me e Revolut.me già compilati con importo e valuta; i rimborsi restano tracciati come transazioni a sé.

Strumenti di lavoro

- **Android Studio** — IDE ufficiale. Java 21, Gradle con Kotlin DSL e version catalog, ViewBinding, Jetpack (Lifecycle, Room, WorkManager), `minSdk 30` / `targetSdk 37`.
- **GitHub** — repository `github.com/AndriMolo/PariPari` per versionamento e coordinamento del team, con commit incrementali per funzionalità sul branch `main`.
- **Room 2.6.1** — SQLite locale: 4 entità, DAO con query reattive LiveData, database alla versione 11 con sette migrazioni esplicite, scritture su un pool di 4 thread.
- **Firestore (BoM 33.10.0)** — *Authentication* (email/password, Google Sign-In, ospite anonimo con account linking); *Cloud Firestore* (collezioni `users`, `groups`, `participants`, `expenses`, `shares`, regole in `firestore.rules`); *Cloud Messaging* (canale notifiche e token di dispositivo); *Hosting* (pagina web di invito con App Links); Storage non è utilizzato.
- **Google ML Kit Text Recognition** — OCR degli scontrini eseguito sul dispositivo, senza inviare immagini a server esterni.
- **ZXing** — generazione dei QR code di invito, con il logo dell'app al centro.
- **Frankfurter API** — tassi di cambio della Banca Centrale Europea, pubblici e senza chiave.
- **Material Components 1.14 (Material 3)** — tema light/dark, BottomSheet, Chip, Card, Snackbar, MaterialDatePicker.
- **JUnit 4** — 14 classi di test unitari (67 test): saldi, conversione valuta, calcolatrice, categorie, quote, codici invito, QR, CSV, avatar, scontrino digitale, preferenze.

1. Funzionalità

L'app è organizzata in quattro schermate raggiunte dalla barra di navigazione inferiore di `MainActivity` — **Schede**, **Saldi**, **Valuta**, **Account** — più la schermata di dettaglio di una scheda (`DettaglioSchedaActivity`), a sua volta divisa in tre sezioni.



Mapa di navigazione delle schermate

1.1 Account e profilo AccountFragment · AccountGuestFragment · AccountProfileFragment ·

`AccountViewModel`

- **Modalità ospite.** Al primo avvio l'app crea un account anonimo Firebase: tutte le funzioni sono subito disponibili e il nome locale è modificabile.
- **Registrazione, accesso e Google.** Pulsanti separati "Crea account" e "Accedi" aprono il form (email validata, password di almeno 6 caratteri), affiancati da "Accedi con Google". Chi era ospite viene promosso ad account registrato con *account linking* (`linkWithCredential`): schede e spese create da ospite restano sue.
- **Verifica e recupero.** Email di verifica con polling ogni 2,5 secondi che porta il badge da "Email non verificata" a "Email verificata"; reinvio della verifica e reset della password via email.
- **Profilo.** Immagine del profilo presa dall'account Google per chi accede con Google, e in ogni caso sostituibile con un'emoji su sfondo colorato o con l'iniziale del nome (`BottomSheetAvatarUtente`: cinque categorie di emoji, dodici colori). Non si carica nessuna immagine: l'avatar è salvato come testo e mostrato in tutte le schede; nome propagato a tutte le schede (`aggiornaNomeUtenteInTuttiIGruppi`); cambio email con riautenticazione.
- **Pagamenti e preferenze.** Handle PayPal e Revolut propagati ai gruppi; valuta predefinita per le nuove schede, lingua tra italiano, inglese, spagnolo, francese e tedesco (applicata a caldo) e tema chiaro, scuro o di sistema.

1.2 Schede e inviti SpeseFragment · SpeseViewModel · NuovaSchedaBottomSheet ·

UniscitiSchedaBottomSheet

- **Elenco.** Le schede in ordine di ultimo aggiornamento, con icona del gruppo, titolo, data, valuta e numero di membri (conteggi da un'unica query aggregata); empty state quando non ce ne sono. Lo stato della schermata è un oggetto immutabile (`SpeseUiState`).
- **Nuova scheda.** BottomSheet con la valuta preimpostata sulla preferenza dell'utente e l'icona della scheda scelta tra quindici emoji rapide con colore di sfondo; il creatore compare come primo membro "(io)", non modificabile in questa fase, e gli altri si aggiungono digitando il nome, come chip rimovibili.
- **Unisciti con codice.** Inserendo il codice a 6 caratteri (o aprendo link e QR code) l'app mostra l'anteprima dei membri della scheda: quelli già associati a un account sono in grigio, quelli inseriti "a mano" sono selezionabili per **subentrare** e diventarne il titolare; in coda c'è sempre l'opzione per aggiungersi come nuovo membro. La scheda viene poi importata in Room e messa in ascolto in tempo reale.
- **Importa da CSV.** Un file CSV esportato da PariPari ricrea scheda, membri, spese e quote sul dispositivo.
- **Gesti.** Pressione lunga e trascinamento per riordinare; swipe a sinistra per eliminare con Snackbar "Annulla", bloccato da `haSaldiInSospeso()` se l'utente ha ancora crediti o debiti aperti.

1.3 Dettaglio scheda DettaglioSchedaActivity · DettaglioSchedaFragment ·

DettaglioSchedaViewModel

- **Toolbar.** Ricerca as-you-type; esportazione in **CSV** (UTF-8 con BOM) o **PDF** con riepilogo KPI, bilanci dei membri e tabella delle spese, condivisi via `FileProvider`; **codice gruppo** con QR code e link `paripari-app-2026.web.app/join?code=...`, che apre l'app direttamente o, dal browser, una pagina web di invito; rinomina; **icona del gruppo** cambiabile in un'emoji su sfondo colorato (`BottomSheetIconaGruppo`); **Lascia scheda**, che rende il membro un "ex membro" (riattivabile) e, se esce il creatore, passa la proprietà al successivo.
- **Sezione Spese.** Elenco raggruppato per data con chip per categoria; un tap apre il dettaglio (`DettaglioSpesaFragment`) con le quote, lo scontrino digitale, modifica ed eliminazione.
- **Sezione Saldi.** Trasferimenti calcolati da `CalcolatoreSaldi`, bilancio netto di ogni membro e accesso allo storico.
- **Sezione Membri.** Attivi in cima con il capogruppo per primo, ex membri in una sezione a comparsa; modifica del nome inline (del proprio, o di tutti per il capogruppo); rimozione consentita al capogruppo o su di sé e bloccata se il membro ha spese o debiti (`haSpese0Debiti`).

1.4 Spese e quote BaseSpesaFragment · NuovaSpesaFragment · ModificaSpesaFragment ·

SpesaUiHelper

- **Campi.** Titolo; importo con calcolatrice inline (12,50+8 diventa 20,50 alla conferma); valuta della singola spesa, anche diversa da quella del gruppo; data da `MaterialDatePicker`; categoria; pagatore scelto tra i membri.
- **Categoria suggerita.** Predefinita "Altro"; mentre si digita il titolo, `CategoriaUtil` propone la categoria ("Esselunga" → Spesa, "Benzina" → Trasporti) e l'emoji corrispondente. Nove categorie: Cibo, Spesa, Trasporti, Alloggio, Svago, Shopping, Bar & Aperitivi, Salute, Altro.
- **Scontrino.** Foto o immagine dalla galleria: l'OCR di ML Kit, eseguito sul dispositivo, estrae totale, data, esercente e le singole voci e precompila il form. Alla spesa resta allegato lo **scontrino digitale** (`ScontrinoDigitale`: voci, totale e differenza rispetto all'importo), mentre la foto viene eliminata subito dopo la lettura: nessuna immagine lascia il telefono.
- **Divisione.** Equa tra i partecipanti selezionati, per percentuali o per parti, con auto-bilanciamento delle quote residue e feedback aptico sulle azioni.
- **Consistenza storica.** Se il pagatore o uno dei partecipanti ha lasciato il gruppo, la modifica è bloccata con un avviso: i saldi passati restano corretti.

1.5 Saldi, rimborsi e pagamenti SaldiFragment · InvioPagamentoBottomSheet ·

StoricoSaldiFragment · DettaglioRimborsoFragment

- **Riepilogo.** Due card, "Ti devono" (verde) e "Devi tu" (rosso), con i totali su tutte le schede e un tap per filtrare; sotto, i bilanci netti. Le spese in valuta diversa sono convertite nella valuta del gruppo con i tassi BCE.
- **Pagamento.** Toccando una voce si apre il foglio di pagamento, che riconosce se l'utente è debitore o creditore e, se il creditore ha un handle, genera il link PayPal.me o Revolut.me con importo e valuta: apribile, copiabile o condivisibile.
- **Rimborsi.** "Segna come saldato", o un rimborso inserito a mano, registra una transazione di tipo *Rimborso* — mittente, destinatario, importo, valuta, data, nota — con schermate di dettaglio e modifica; è esclusa dal totale speso e riduce il debito in locale e su Firestore.
- **Storico.** Schermata dedicata con filtro per scheda; un pareggio si può annullare, salvo quando coinvolge un membro uscito.

1.6 Notifiche FirestoreSyncManager · PariPariMessagingService · SyncNotificheWorker

- **Movimenti del gruppo.** Quando un altro membro aggiunge o modifica una spesa, o segna un rimborso, il listener Firestore che riceve la modifica genera una notifica nativa ("Nuova spesa in ...", con importo e propria quota); il tap apre la scheda interessata.
- **Permessi e canale.** All'avvio l'app chiede il permesso `POST_NOTIFICATIONS` (Android 13+) e registra il canale di notifica.
- **Ad app chiusa.** `SyncNotificheWorker` (WorkManager, ogni 15 minuti quando c'è rete) controlla su Firestore le spese aggiunte dagli altri membri e notifica quelle non ancora viste, senza duplicati grazie al confronto con Room.
- **Firestore Cloud Messaging.** Il servizio riceve eventuali messaggi push e registra il token del dispositivo in `users/{uid}/fcm_tokens`: la base per notifiche inviate dal server in tempo reale.

1.7 Convertitore valuta ValutaFragment · ValutaViewModel · CurrencyRepository

- **Tassi BCE.** `CurrencyRepository` interroga `api.frankfurter.dev` al massimo una volta ogni 24 ore e salva i tassi in `SharedPreferences`; senza rete usa la cache o i tassi di fallback integrati. 30 valute supportate, con bandiera e nome. Gli stessi tassi servono a convertire le spese multivaluta nei saldi.
- **Conversione as-you-type.** Risultato e tasso applicato si aggiornano mentre si digita; il pulsante "Inverti" scambia le due valute.
- **Selettore.** BottomSheet con le valute frequenti in testa e ricerca testuale sul catalogo completo; lo stesso selettore è usato per la valuta di una scheda o di una spesa.

2. Architettura software

2.1 Pattern architetturale (Clean Architecture / MVVM)

PariPari adotta il pattern **MVVM** (Model-View-ViewModel) raccomandato dalle linee guida Android, organizzato secondo i principi della **Clean Architecture**: il codice è diviso in layer con una sola regola per le dipendenze, che vanno sempre dall'alto verso il basso. Le schermate conoscono solo i ViewModel, i ViewModel solo i Repository, e nessun layer sa nulla di chi lo usa. Le *View* sono passive: mostrano lo stato che ricevono e inoltrano le azioni dell'utente; i *ViewModel* espongono lo stato come `LiveData` e sopravvivono alle rotazioni; i *Repository* sono l'unica porta verso i dati e nascondono ai livelli superiori se un'informazione arriva dalla cache locale o dal cloud.

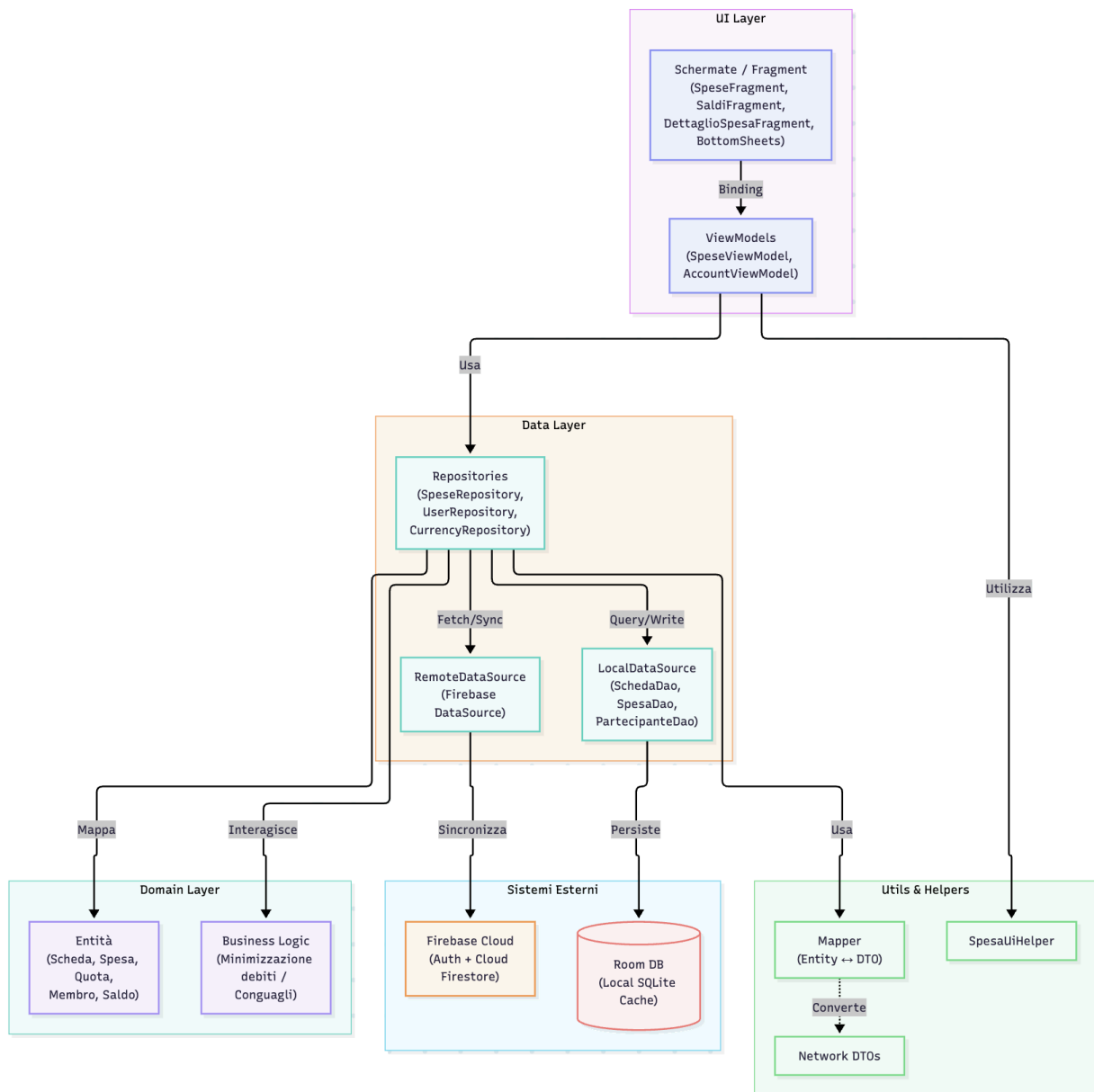
Ai tre layer canonici — **UI Layer**, **Domain Layer** e **Data Layer** — si affiancano il blocco trasversale **Utils & Helpers** (Mapper, DTO di rete e helper di presentazione) e i **Sistemi esterni** (Room e Firebase), trattati come dettagli implementativi sostituibili senza toccare la logica. Il pattern è realizzato in Java con le librerie Android Jetpack (ViewModel, LiveData, Room, WorkManager) e senza framework di dependency injection: Repository e database sono singleton creati da `PariPariApplication` all'avvio.

La scelta che più caratterizza l'architettura è l'approccio **offline-first**: Room è l'unica fonte di verità della UI, ogni scrittura avviene prima in locale e la sincronizzazione con Cloud Firestore è un processo in background che parte quando rete e utente autenticato sono disponibili (§2.4 e §2.5).

Tre conseguenze concrete di questa impostazione attraversano tutto il progetto. La **testabilità**: la logica di business e gli helper non dipendono da Firebase né dalle View, e sono coperti da test JUnit eseguiti sulla JVM senza emulatore. La **sostituibilità**: Room e Firestore sono confinati dietro Repository e sorgenti dati, così cambiare backend o aggiungere una sorgente (come è successo con l'API dei cambi) non tocca le schermate. La **robustezza al ciclo di vita**: lo stato vive nei ViewModel e nei `LiveData`, sopravvive alla rotazione e ai ricreamenti delle Activity e non lascia riferimenti pendenti alle View.

2.2 Diagramma architetturale dei layer

Il diagramma mostra i cinque blocchi e le loro relazioni. Le frecce vanno sempre verso il basso: *Binding* è il legame tra schermate e ViewModel tramite ViewBinding e `LiveData`; *Usa* e *Utilizza* sono le dipendenze dirette dei ViewModel dai Repository e da `SpesaUiHelper`; *Query/Write* e *Fetch/Sync* distinguono la sorgente locale (letture e scritture immediate) da quella remota (invio dei pendenti e ascolto in tempo reale); *Persiste* e *Sincronizza* collegano le due sorgenti ai sistemi esterni; *Mappa* e *Interagisce* indicano che il Repository lavora sulle entità di dominio e invoca la Business Logic; *Converte*, tratteggiata, è la trasformazione delle entità in DTO di rete fatta dal Mapper. Le sezioni che seguono descrivono ogni blocco con le classi che lo realizzano nel codice.



Architettura a layer: UI, Domain, Data, Utils & Helpers e Sistemi esterni; le frecce indicano la direzione delle dipendenze

2.2.1 UI Layer

Schermate / Fragment — due Activity (`MainActivity`, con la barra di navigazione, e `DettaglioSchedaActivity`), quattordici Fragment (`SpeseFragment`, `SaldiFragment`, `ValutaFragment`, `account`, `dettaglio scheda e spesa`, `nuova e modifica spesa da BaseSpesaFragment`, `storico e rimborsi`), sei BottomSheet (`nuova scheda`, `unisciti`, `pagamento`, `valuta`, `avatar`, `icona gruppo`) e nove Adapter. Sono viste passive, legate ai layout XML tramite **ViewBinding**: osservano i `LiveData` dei `ViewModel` e inoltrano le azioni dell'utente.

ViewModels — `SpeseViewModel`, `DettaglioSchedaViewModel`, `AccountViewModel` e `ValutaViewModel` estendono `AndroidViewModel`: espongono solo `LiveData`, combinano più sorgenti con `MediatorLiveData` e `Transformations`, rappresentano lo stato della schermata con oggetti immutabili (`SpeseUiState`) e usano i `Repository` come unica porta verso i dati.

2.2.2 Domain Layer

Entità — `Scheda`, `Spesa`, `Partecipante` (il membro), `SpesaPartecipante` (la quota) e `RisultatoSaldi` con `TrasferimentoSaldo` (il saldo), più `ScontrinoDigitale` e i modelli di supporto per le liste. Sono il vocabolario condiviso da tutti i layer: le stesse classi sono annotate come entità Room, così che non esistano due versioni del modello da tenere allineate.

Business Logic — `CalcolatoreSaldi` concentra la regola di business centrale, la **minimizzazione dei debiti**. Per ogni scheda, dopo aver ricondotto alla valuta del gruppo le spese in altre valute (con il tasso congelato al momento dell'inserimento), lavora in tre passi:

1. **Bilancio netto** di ogni membro: somma degli importi che ha anticipato, meno le quote residue che deve (`quota - quota_pagata`); i rimborsi già registrati riducono sia il debito sia il credito.
2. **Separazione** in debitori (bilancio negativo) e creditori (bilancio positivo), con arrotondamento al centesimo.
3. **Accoppiamento greedy**: il primo debitore paga al primo creditore il minimo tra debito e credito; chi si azzerava avanza. Ogni trasferimento chiude almeno una posizione, quindi i conguagli sono al più $N - 1$ per N membri.

Esempio: Anna ha anticipato 90 €, Bruno 30 € e Carla 0 € su spese divise equamente (40 € a testa). Bilanci: Anna +50, Bruno -10, Carla -40. Invece di tre rimborsi incrociati bastano due trasferimenti: Bruno → Anna 10 € e Carla → Anna 40 €.

2.2.3 Data Layer

Repositories — tre repository, uno per area: `PariPariRepository` per schede, spese, membri, saldi e inviti (lo *SpeseRepository* del diagramma), `UserPreferencesRepository` per profilo e preferenze — lingua, tema, avatar, valuta predefinita, ordinamento delle schede — (lo *UserRepository*) e `CurrencyRepository` per i tassi di cambio. Sono singleton: coordinano le due sorgenti dati, invocano la Business Logic e pubblicano i risultati come `LiveData`.

LocalDataSource — `AppDatabase` con i DAO Room `SchedaDao`, `SpesaDao` e `PartecipanteDao`: query e scritture locali (*Query/Write*) che persistono nella cache SQLite. È la sorgente che alimenta la UI.

RemoteDataSource — il *Firebase DataSource* è `FirestoreSyncManager`: invio dei dati pendenti, listener in tempo reale, migrazione dei dati dell'ospite all'account, anteprima e ingresso nei gruppi con codice (*Fetch/Sync*). Si appoggia a `NetworkConnectivityMonitor` per sapere quando può lavorare.

2.2.4 Utils & Helpers

Mapper — la conversione tra entità e documenti remoti sta nei metodi di caricamento e lettura di `FirestoreSyncManager`: in uscita ogni entità diventa un `Map<String, Object>` con i soli campi da condividere, in entrata ogni `DocumentSnapshot` torna entità; UI e logica non conoscono il formato dei documenti. **Network DTOs** — i documenti Firestore e il JSON dell'API dei cambi (§2.5.1).

SpesaUiHelper — validazione dell'importo, auto-bilanciamento e ripartizione delle quote, formattazione. Completano il blocco le utility di `util`: categorie, codici invito e QR, calcolatrice, OCR, avatar, CSV, PDF, Snackbar, feedback aptico.

2.2.5 Sistemi esterni

Room DB — il database SQLite locale (versione 11, quattro tabelle, sette migrazioni), cache persistente e fonte di verità offline. **Firestore** — *Authentication* (email, Google, ospite) e *Cloud Firestore* (sincronizzazione in tempo reale nel gruppo), più *Cloud Messaging* e *Hosting*. *Storage* non è utilizzato: avatar e icone sono emoji o iniziali su sfondo colorato salvate come testo, gli scontrini restano dati strutturati e l'unica immagine remota è la foto del profilo Google, letta dall'URL dell'account. Fuori da Firebase c'è solo l'API pubblica **Frankfurter** per i cambi.

2.3 Struttura dei package

Il codice vive nel package `com.example.paripariapp`: 78 classi Java (circa 20.000 righe) organizzate per responsabilità, così che ogni package corrisponda a un blocco del diagramma; alla radice restano `MainActivity` e `PariPariApplication`, che all'avvio crea il Repository e programma il worker delle notifiche.

ui.view

UI Layer · Schermate

2 Activity · 14 Fragment · 6 BottomSheet
9 Adapter · SpesaUiHelper (31 classi)

ui.viewmodel

UI Layer · ViewModels

SpeseViewModel · DettaglioSchedaViewModel
AccountViewModel · ValutaViewModel
SpeseUiState (stato immutabile)

data.model

Domain Layer · Entità

Scheda · Spesa · Partecipante
SpesaPartecipante · RisultatoSaldi
TrasferimentoSaldo · ScontrinoDigitale
SyncStatus · modelli per le liste (12)

data.repository

Data Layer · Repositories

PariPariRepository
UserPreferencesRepository
CurrencyRepository

data.local

Data Layer · LocalDataSource

AppDatabase (Room, versione 11)
SchedaDao · SpesaDao · PartecipanteDao

data.remote

Data Layer · RemoteDataSource

FirestoreSyncManager
(sync, listener in tempo reale,
Mapper entità ↔ documenti)

util

Business Logic · Utils & Helpers

CalcolatoreSaldi · SpesaUtil
ImportoUtil · CategoriaUtil
CodiceInvitoUtil · QrCodeUtil
ScontrinoOcrUtil · AvatarVisualUtil
ImportatoreCsvUtil · EsportatoreDati ...
(18 classi)

service

Sistemi esterni · FCM e WorkManager

PariPariMessagingService
SyncNotificheWorker

Fuori dal codice Java: `res/layout` con i layout XML letti da `ViewBinding`; `res/values` con le stringhe in italiano e le traduzioni `values-en`, `-es`, `-fr`, `-de`, oltre a `values-night` per il tema scuro; `app/src/test` con 14 classi JUnit (67 test) sulla logica indipendente da Android. Alla radice del repository stanno la configurazione Firebase (`firebase.json`, `firestore.rules`, `firestore.indexes.json`), la pagina web di invito in `public/` e la cartella `functions/`, predisposta per future Cloud Function.

2.4 Gestione dati e persistenza

2.4.1 Database locale e caching (Room, SQLite)

Quattro tabelle Room (`AppDatabase`, versione 11), con ID `UUID` generati dal client così da poter creare dati offline senza conflitti; ogni riga porta un `sync_status` (`SYNCED`, `PENDING_INSERT`, `PENDING_UPDATE`, `PENDING_DELETE`) che funge da coda di invio verso il cloud.

schede	partecipanti
PK id titolo · descrizione · icona_url (v9) valuta_predefinita · creatore_id data_creazione · data_aggiornamento codice_invito (v6) · sync_status	PK id FK scheda_id → schede nome · email · photo_url (v9) user_id · previous_user_id · stato (v8) paypal_handle · revolut_handle (v7) sync_status
spese	spese_partecipanti
PK id FK scheda_id → schede titolo · importo · valuta · data_spesa categoria · pagato_da_id → partecipanti scontrino_url · scontrino_json (v10) tasso_cambio (v11) · sync_status	PK,FK spesa_id → spese PK,FK partecipante_id → partecipanti quota · quota_pagata (v5) sync_status

Una scheda ha N partecipanti e N spese; ogni spesa è pagata da un partecipante e ripartita tramite la tabella ponte, che conserva quota dovuta e quota già saldata. `icona_url` e `photo_url` contengono la descrizione testuale dell'icona (`emoji:🍷:#1976D2`, `initial:R:#3F51B5`) oppure l'URL della foto Google; `scontrino_json` è lo scontrino digitale letto dall'OCR; `tasso_cambio` il cambio congelato al momento della spesa. **Migrazioni:** 4→5 `quota_pagata`; 5→6 `codice_invito`; 6→7 handle PayPal e Revolut; 7→8 `user_id`, `previous_user_id`, `stato`; 8→9 `icona_url` e `photo_url`; 9→10 `scontrino_json`; 10→11 `tasso_cambio`. Ogni evoluzione preserva i dati già sul dispositivo.

Accesso e caching. I DAO espongono 53 query, di cui 20 reattive (`LiveData`) che aggiornano la UI a ogni modifica; le scritture passano da un pool dedicato di quattro thread (`databaseWriteExecutor`) e le operazioni composte sono `@Transaction`. Room è la cache persistente e la fonte di verità: la UI legge solo da qui, anche quando i dati arrivano dal cloud. Due cache più piccole la affiancano: `SharedPreferences` per le preferenze utente e per i tassi di cambio (validi 24 ore) e una `LruCache` in memoria per le foto profilo Google (`ImageLoaderUtil`).

2.4.2 Servizi cloud e Backend-as-a-Service (Firebase)

Authentication — al primo avvio l'app crea un utente anonimo (modalità ospite); registrazione email/password e Google Sign-In promuovono l'ospite con `linkWithCredential`, così l'`uid` e i dati restano gli stessi. Verifica dell'email, cambio email con riautenticazione e reset della password usano le API native di Firebase Auth.

Cloud Firestore — è il backend condiviso del gruppo. Ogni scheda è un documento `groups/{id}` (titolo, valuta, creatore, codice invito, icona e l'elenco `membriUids` usato dalle regole di sicurezza) con le sotto-collezioni `participants`, `expenses` ed `expenses/{id}/shares`; ogni utente ha un documento `users/{uid}` con la sotto-collezione `fcm_tokens`. Le regole in `firestore.rules` richiedono un utente autenticato e riservano le modifiche strutturali al creatore. Non c'è un server applicativo: i client leggono e scrivono direttamente e i listener propagano le modifiche al gruppo.

Cloud Messaging e WorkManager — `PariPariMessagingService` registra il token del dispositivo e mostra i push in arrivo; `SyncNotificheWorker`, programmato ogni 15 minuti con vincolo di rete, controlla ad app chiusa le spese aggiunte dagli altri e genera le notifiche mancanti. **Hosting** — serve la pagina `paripari-app-2026.web.app/join` e l'`assetlinks.json` che apre i link di invito direttamente nell'app.

2.5 Networking e integrazione API esterne

2.5.1 Definizione API / Endpoint e DTO

- **Cloud Firestore** (SDK Firebase; non c'è un backend REST proprio). Gli "endpoint" sono i percorsi dei documenti — `groups/{id}`, `.../participants/{id}`, `.../expenses/{id}`, `.../shares/{id}`, `users/{uid}` — e i DTO sono i documenti stessi: `Map<String, Object>` con campi in camelCase (`valutaPredefinita`, `codiceInvito`, `membriUids`, `pagatoDaId`, `tassoCambio`...). Il Mapper traduce le colonne Room in snake_case e non invia i campi solo locali come `sync_status`.
- **Frankfurter API** — `GET api.frankfurter.dev/v1/latest?from=EUR`, senza chiave: JSON `{ base, date, rates: {...} }` letto con `org.json` in `CurrencyRepository` e tenuto in memoria e in `SharedPreferences`; 30 valute supportate.
- **Link esterni** — pagamenti con URL `paypal.me/{handle}/{importo}` e `revolut.me/{handle}`; inviti con `paripari-app-2026.web.app/join?code=...`, che l'App Link porta a `UniscitiSchedaBottomSheet`.

2.5.2 Gestione asincronia, errori e resilienza di rete

Asincronia. Il progetto è in Java, senza coroutine: le chiamate Firebase restituiscono `Task` gestiti con `addOnSuccessListener` / `addOnFailureListener`, le letture continue usano `addSnapshotListener`, scritture Room e chiamate HTTP girano su executor dedicati (`databaseWriteExecutor`, un thread per i cambi) e i risultati tornano al main thread con `postValue` o un `Handler` sul `Looper` principale. I flussi a più passi espongono callback dedicate (`OnJoinSchedaCallback`, `OcrCallback`...); le scritture correlate — scheda con membri, spesa con quote — viaggiano in un unico `WriteBatch` atomico.

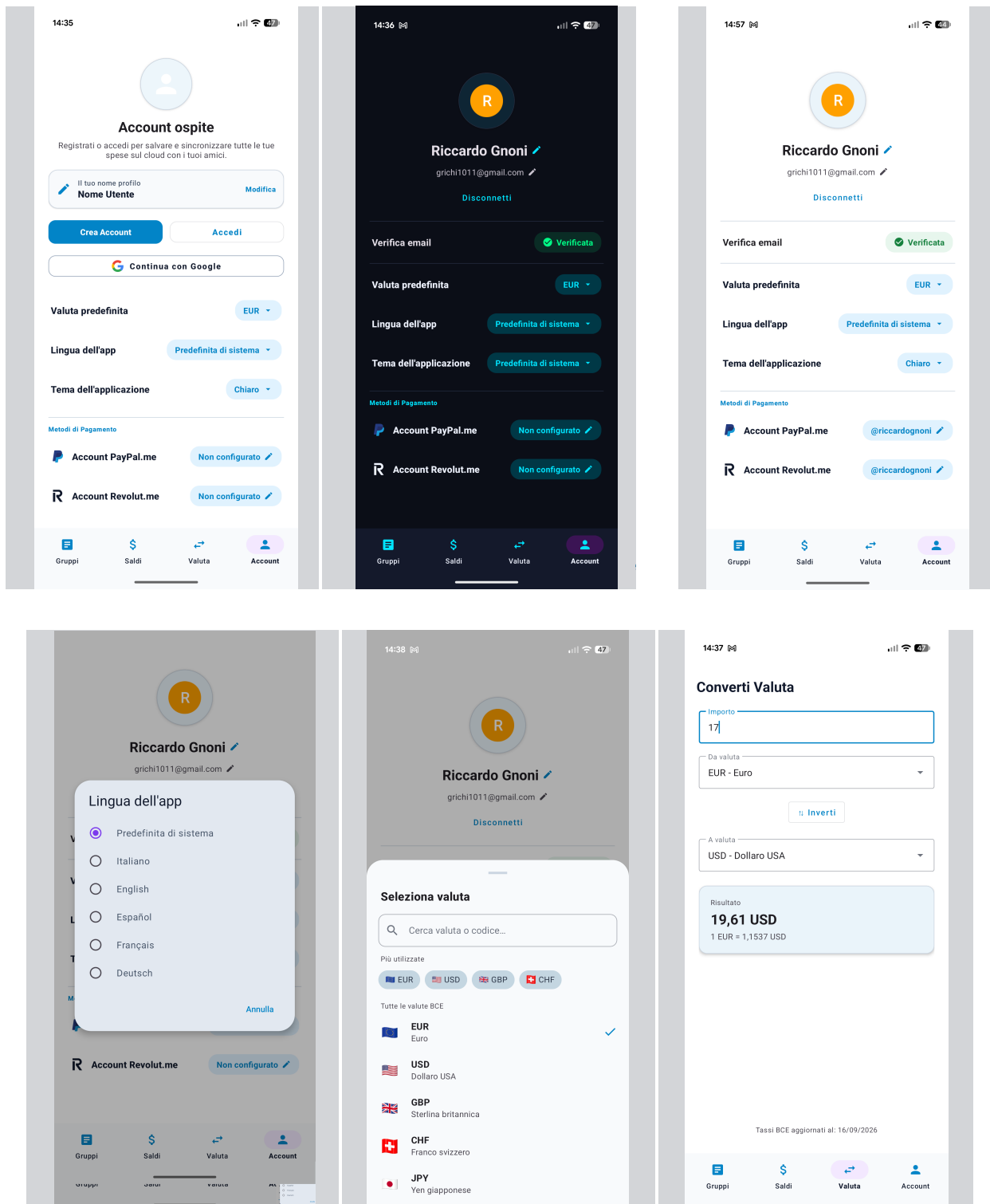
Resilienza di rete. `NetworkConnectivityMonitor` decide se la rete c'è: al ritorno della connessione invia le righe pendenti (`syncPendingData`) e riattiva i listener (`startRealtimeSync`); senza rete l'app lavora su Room e nulla va perso, perché ogni scrittura è già in locale. L'app sospende i listener in background e li riattiva in primo piano; il worker ha il vincolo `NetworkType.CONNECTED`; la chiamata HTTP ha timeout di 8 secondi e in caso di errore restano cache e tassi di fallback; le operazioni che richiedono un utente passano da `ensureAuthenticatedSession`, che crea al volo la sessione ospite.

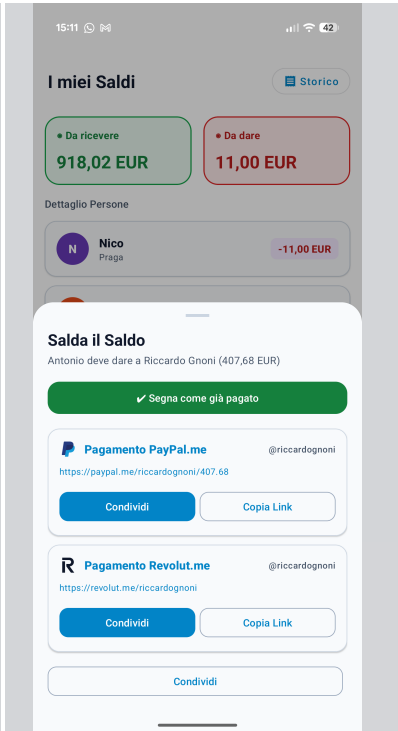
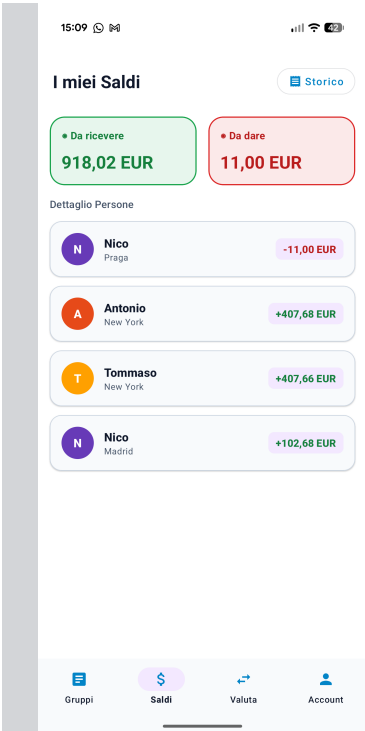
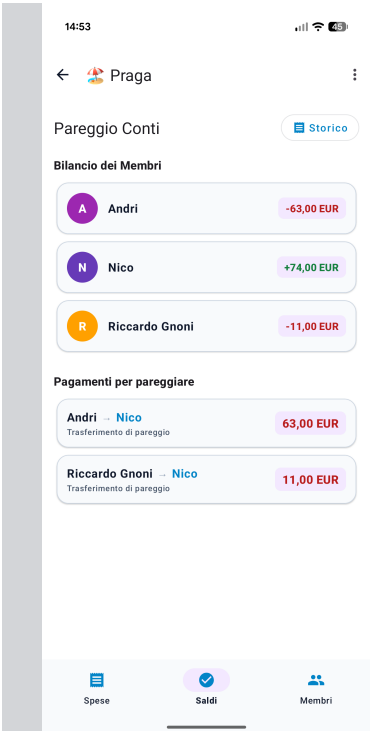
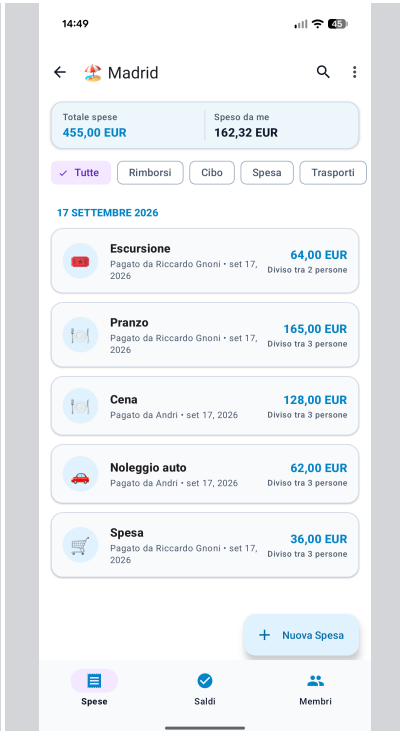
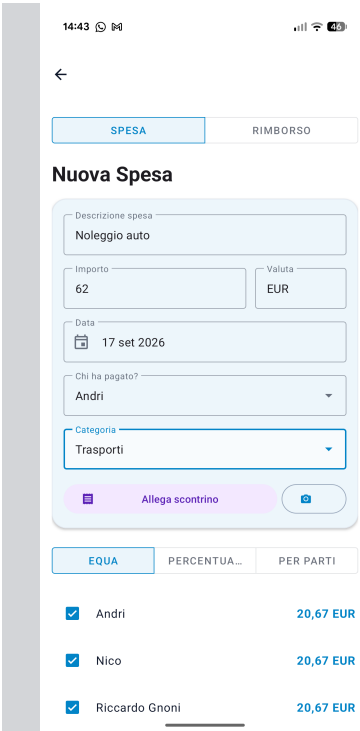
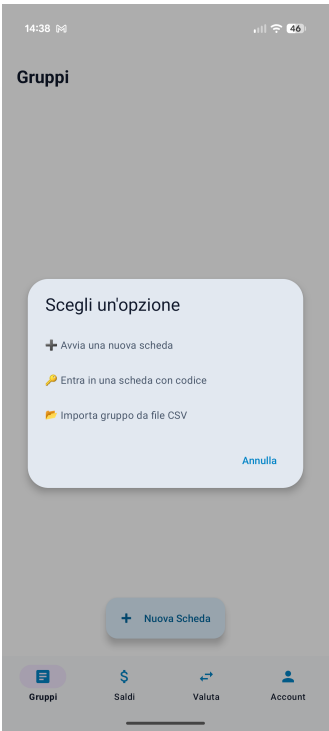
Errori. Ogni fallimento remoto è loggato e arriva alla schermata come `onError`, mostrato con `AppSnackbar`; `CurrencyRepository` espone `errorMessage` e `isLoading` come `LiveData`. Non ci sono `Interceptor` perché non si usa `OkHttp/Retrofit`: il loro ruolo trasversale — connettività e sessione autenticata prima di ogni chiamata — è svolto dal monitor di rete e dal guard di autenticazione.

3. Design

L'interfaccia segue **Material 3** — Card, Chip, BottomSheet, BottomNavigation, Snackbar — con una palette dedicata per tema chiaro e scuro. Layout XML con ViewBinding, edge-to-edge, empty state per ogni lista vuota e feedback aptico sulle azioni.

3.1 Layouts





3.2 Palette e tipografia

La palette è costruita sui ruoli semantici di Material 3. Il tema chiaro usa un ciano profondo su fondi neutri freddi; quello scuro passa a toni neon su sfondo quasi nero. Verde, rosso e arancione sono riservati agli stati contabili: credito, debito, avviso.

#0284C7 #7C3AED #F8FAFC #0F172A #15803D #B91C1C					
Ruolo	Risorsa	Chiaro	Scuro	Uso	
Primario	<code>md_theme_primary</code>	 #0284C7	 #00E5FF	Toolbar, pulsanti, accenti attivi	
Contenitore primario	<code>md_theme_primaryContainer</code>	 #E0F2FE	 #003846	Badge circolari, pillole valuta	
Secondario	<code>md_theme_secondary</code>	 #7C3AED	 #A855F7	Accenti secondari	
Sfondo	<code>md_theme_background</code>	 #F8FAFC	 #0B0E17	Sfondo delle schermate	
Superficie	<code>md_theme_surface</code>	 #FFFFFF	 #101422	Card, BottomSheet, dialog	
Credito	<code>credit_green</code>	 #15803D	 #00FF87	"Ti devono", saldo chiuso	
Debito	<code>debt_red</code>	 #B91C1C	 #FF2A6D	"Devi tu", eliminazione	
Avviso	<code>warning_orange</code>	 #D97706	 #FFB800	Email non verificata	

Tipografia e stili

- **Font.** Nessun font incorporato: l'app usa la scala tipografica di Material 3 con il carattere di sistema (Roboto), per coerenza con la piattaforma e massima leggibilità delle cifre.
- **Card** (`Widget_PariPari_CardItem`): angoli a 16dp, elevazione 1dp, bordo di 1dp sul colore outline, sfondo `surfaceContainerLow`.
- **Badge circolari** (`Widget_PariPari_CircleIconBadge`): raggio 24dp su `primaryContainer` per avatar, stati e valute.
- **BottomSheet e Snackbar.** Fogli con angoli superiori a 28dp e maniglia 32×4dp; Snackbar a pillola (20dp) su `surfaceContainerHighest`, con animazione a scorrimento, al posto dei toast.
- **Icona e splash.** Icona adattiva con il glifo "PP" — un più e un meno, le due facce di ogni spesa condivisa — su sfondo bianco, con variante monocromatica per Android 13+; lo stesso glifo anima lo splash screen di sistema.

Conclusioni

PariPari copre l'intero ciclo di una spesa condivisa: fotografarla o registrarla, ripartirla, calcolare chi deve cosa a chi, saldare e avvisare gli altri. L'architettura MVVM con Repository e l'approccio offline-first hanno retto bene alla crescita del progetto: le funzionalità aggiunte nella seconda fase — OCR, inviti web e QR, rimborsi, notifiche, avatar e icone — si sono innestate senza cambiare l'impianto.

Punti di forza

1. **Funziona sempre.** Nessuna operazione dipende dalla rete: Room risponde subito e la sincronizzazione recupera il pregresso quando torna la connessione, in modo trasparente per l'utente e con la pausa in background a tutela della batteria.
2. **Conti chiari.** Bilanci netti in un'unica valuta anche con spese miste, trasferimenti minimi, rimborsi tracciati come transazioni con uno storico annullabile e blocchi che impediscono di lasciare debiti orfani.
3. **Inserimento rapido.** Scontrino fotografato e letto sul dispositivo, calcolatrice nell'importo, categoria suggerita dal titolo, divisione per percentuali o parti che si bilancia da sola.
4. **Dal saldo al pagamento in un gesto.** Il link PayPal.me o Revolut.me precompilato chiude il cerchio, e la notifica avvisa il gruppo senza scambiarsi messaggi.

Ostacoli incontrati

1. **Sincronizzazione bidirezionale.** Tenere allineati Room e Firestore senza duplicare scritture ha richiesto uno stato per riga e una gestione attenta del ciclo di vita dei listener, isolata nella sorgente dati remota e sospesa quando l'app non è in primo piano.
2. **Identità dei membri.** Un partecipante può nascere come semplice nome digitato da un altro utente e solo dopo corrispondere a un account: il modello con `user_id`, `previous_user_id` e `stato`, il subentro all'ingresso e l'account linking per gli ospiti hanno risolto i casi di "chi sono io" in ogni scheda.
3. **Integrità dei saldi.** Eliminare una scheda o un membro con pendenze aperte corrompeva i conti: i controlli `hasaldiInSospeso()` e `haspese0Debiti()` bloccano le operazioni pericolose, la modifica di una spesa è inibita se un partecipante coinvolto è uscito e i membri usciti restano come "ex" invece di sparire.
4. **Evoluzione dello schema.** Pareggi parziali, codici invito, handle di pagamento, subentro, icone e avatar, scontrino digitale e tasso di cambio congelato sono arrivati in fasi successive: sette migrazioni Room esplicite hanno permesso di aggiungerli senza perdere i dati sul dispositivo.